University of Nottingham
UK | CHINA | MALAYSIA

Anura: A GUI Debugger

Submitted April 2026, in partial fulfilment of the conditions for the award of the degree BSc Computer Science.

20570759
School of Computer Science
University of Nottingham

I hereby declare that this dissertation is all my own work, except as indicated in the text:

Signature JC
Date 10/04/26

I hereby declare that I have all necessary rights and consents to publicly distribute this dissertation via the University of Nottingham's e-dissertation archive.

1 Abstract

This dissertation explores the creation of a new debugger, Anura, developing both a backend and frontend. It first discusses existing debuggers and the features that Anura takes from them. It then gives background on the target architecture and OS that Anura currently implements, before giving implementation details for each of the features of a basic debugger. The dissertation then introduces a custom language called the (I)nstruction (S)et (D)escription (L)anguage which aims to make the creation of disassemblers easier for one of Anura's goals of extensibility. Following this are two features designed to help with data flow analysis, the first related to time-travel debugging where stack frames, variable values, and control flow are saved and navigable by the user. The second takes a target variable and value and, using a new section of compiler information, propagates this target value through the program, so that the debugger can stop at the earliest point that it can guarantee the variable will later evaluate to that target value.

The findings from this dissertation show that data analysis through propagating a target value can be complicated by common programming techniques such as pointers and function arguments, but that it can provide more insight to a developer as to why the program is failing with certain values. It also shows ISDL to be a powerful enough language to describe all of the x64 instructions, and is extensible to support languages such as ARM64.

Contents

2 Introduction

2.1 Aims

The overall aim of this project is to develop an extensible GUI debugger that is independent of the current GDB and LLDB dominated backend choices. Providing users with a user interface that displays compiler generated debug information. As well as, creating a backend that can be extended to support many different target operating systems and instruction sets.

There are three sub-aims for the project. The first is to develop an instruction set disassembly language (ISDL) that can be used to generate disassemblers. The second is to create a form of stack frame saving that saves the data and control flow points of the program to later be walked by the user, referred to as break-save. The third is to explore and create a demonstration of a feature that allows users to specify target values for a variable and for the debugger to be able to use compiler information to propagate this value down the program. This allows the debugger to break at the earliest point in execution that it can guarantee the variable will later evaluate to the target value, this is referred to as break-on-cause.

2.2 Motivation

Debuggers are used throughout all aspects of software development, and they have real impact on the efficiency (and enjoyment) of programming. There is still much more room for debuggers to grow, especially within data analysis. This is the motivation behind the break-save and break-on-cause features which both help reason about the data flow of a program.

Developing a multi-target debugger comes with lots of new work for each target, one of the reasons for developing ISDL is to make adding disassembly support for each target a much faster process. The goal of ISDL therefore is to allow writing of instructions close to how they are written in instruction set manuals, so that little translation is required from the developer.

3 Background

3.1 What is a debugger

A debugger is a program designed to aid developers working on a program, helping them understand what their program is doing at runtime. Debuggers primarily help in finding logic errors, which are the largest cause of software bugs. Where syntax errors can be caught by compilers, logic errors rely on code reviews and a deep understanding of how the code will execute. A debugger does this by providing a view into the code, data, and control flow of a program, both statically, and dynamically as the program is run. It uses the debug information provided by compilers to show information such as variable names and types, a stack trace of the current execution, and a link between source code lines and assembly instruction lines for placing breakpoints. It also provides a way of stopping at a particular point in the program so that the developer can inspect a slice of the execution that is causing the issue.

3.2 Debugging: A history

The origin of the word debugging in computer science is largely attributed to Grace Hopper, who gave an anecdote in Info World's October 5th 1981 edition. "In 1945, while working in a World War 1 vintage non-air-conditioned building on a hot, humid summer day, the computer stopped. We searched for the problem and found a failing relay – one of the big signal relays. Inside we found a moth that had been beaten to death. We pulled it out with tweezers and taped it to the log book. From then on, when the officer came in to ask if we were accomplishing anything, we told him we were 'debugging' the computer." [1] Debugging was also used as a term in other science and

engineering areas before this, such as in aeronautics around 1944. [2] Overall the term has been in use for a long time describing the troubleshooting of issues and errors.

Debugging has evolved along side all aspects of computing, from hardware oriented to reading process memory, to operating system apis. Through this development processes have become more protected from other parts of the computer, with memory access restrictions, but these restrictions have also produced much more robust and wide ranging OS API support.

For a time when computers took up a room and programs inputted through punched cards there was little debugging capability that the computer provided. The cost of running another program to debug another was unfeasible, and so the burden fell mostly on the developer, to instead not create bugs, or to debug by hand.

As systems grew more powerful specific debugging tools were developed, for example (D)ynamic (D)ebugging (T)echnique a series of debugging tools were made for the PDP-1 [3]. DDT was an interactive debugger that allowed for symbolic debugging where the variable and function names would be displayed instead of their mapped addresses. This also included breakpoints allowing the execution of the program to be halted temporarily.

Later systems also had hardware level debugging, for example the PDP-11 has a series of control switches that allowed for examining data at an address, poking data into an address, continuing execution, as well as single stepping an instruction or even single cycling the cpu [4]. This was interfaced through a series of switches on the peripheral of the computer. Data and addresses could be inputted into a special register called the switch register, which was made of a series of physical switches representing their bits on the front panel. To examine data at an address for example you would input the address in the switch register, then select LOAD to load the switch register into the bus address register, and then finally use the EXAM switch to read the data at the address.

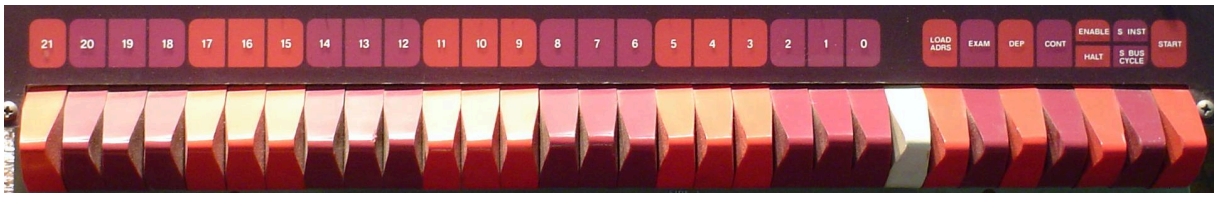

Figure 1: PDP-11 front panel by Dave Fischer under the CC BY-SA 3.0 license [via flickr] (cropped)

As the main programming level moved from assembly to high-level languages debuggers expanded to provide source level functions. For example, dbx found on many OSes provided symbolic debugging for languages like C, C++, and Fortran [5]. Dbx also provided source level stepping, the ability to skip over an entire source line.

Within modern debugging development tools like the (G)NU project (D)e(B)ugger were introduced for UNIX [6]. GDB also allows for remote debugging where the target and host can be separate machines that then communicate commands and data to debug [7]. GDB and later LLDB contain all the features of modern debuggers, with source and assembly level interaction, breakpoints, stepping, viewing registers, evaluating expressions, all with support for multiple languages.

Newer developments within debuggers include the (D)ebug (A)dapter (P)rotocol which was designed and developed by microsoft as a protocol to abstract how development tools communicate with debuggers, making it possible to reuse a language's DAP implementation across multiple development tools [8].

Throughout this development the debugger has become a tool that does even more for the developer, removing the tedium of mapping addresses, single stepping through hundreds of lines, or

disassembling by hand. This project hopes to build on this by taking an aspect of debugging that currently requires lots of a developer's time and place the burden on the debugger instead.

3.3 Related work

There are a number of existing debuggers, ISDLs, and data flow analysis algorithms, this section will discuss some of these and what Anura hopes to take from them, improve on, and avoid.

3.3.1 Related debuggers

There exist two main types of debuggers; backend / terminal debuggers and graphical wrapper debuggers. The former provide the actual interface with the target program, being able to control its execution flow, read and parse its debug information, and alter its registers and memory. Graphical wrapper debuggers are a strictly graphical addition to a backend debugger; providing an easier to use interface than the traditional terminal interface.

3.3.1.1 Backend debuggers

The two most common backend debuggers are GDB from the GNU project [6] and LLDB from the LLVM project [9]. They both support many different targets, GDB for example supports around 80 OS-Architecture combinations [10], as well as supporting minor variations that can 'describe themselves' using an XML file with listed changes from the default [11]. LLDB has a slightly smaller supported target list [12], however it does include windows support that is under active development. This feature of not being restricted to one target architecture or OS is something this project aims to take from GDB and LLDB. Both of these debuggers interact with the user through the terminal, however the command structure for each is very different. GDB's commands are fairly random, being developed overtime with no cohesion. LLDB explicitly tries to avoid this [13] and instead formats them as '<noun> <verb> [-options [option-value]] [argument [argument…]]' for example breakpoint set –name foo. Anura plans to take a command structure similar to LLDB over GDB, as this allows users to intuit how to use new commands rather than have to know the specific word.

3.3.1.2 Frontend debuggers

There exists three prominent graphical debuggers; Jetbrains IDEs, VS Code, and WinDbg. WinDbg is one of the main debuggers for windows development, using DbgEng.dll as its backend [14]. Although WinDbg provides a GUI there is still a large amount of interaction that is completed through a terminal in the GUI. Anura plans to have both GUI and terminal interaction, but for terminal interaction to not be as core as in WinDbg.

Jetbrains IDEs and VS code are purely graphical debuggers in that they use GDB or LLDB as their backend debugger and provide the GUI. These provide the ability to click on a source line's edge to place a breakpoint, view the disassembly of a source file, with buttons for the control flow actions such as step-over, step-into, continue, etc. The layout of these differs slightly, Anura takes the most from Jetbrains' design with control buttons in the top right, terminal at the bottom, and source view / disassembly in the middle. This is a design that I found most intuitive and so used as inspiration for Anura.

3.3.2 Related ISDLs

As part of this project I have researched two other description languages, Charlie Brej's CHUMP [15] and Ghidra's SLEIGH [16]. CHUMP is a simpler version of what I would need to design, as it only allows for easily writing reduced instruction sets like ARM32, STUMP16, and MIPS32 [17], and the syntax of the language is very similar to that of LISP, being parenthesis heavy which makes it difficult to read. It does however support both disassembly and assembly, which is not something I plan to support.

Ghidra's SLEIGH is a much more general language as it describes the architecture as a whole [18], not just the format of the instructions and produces information in the form of p-code [19] for both disassembling and de-compiling which is beyond what is required for this project. The output of SLEIGH is more structured, instead of being a string output of just the disassembly, it includes the operands, flags affected, opcode, and base operation split into a structure.

ISDL sits between these two languages allowing for more complex descriptions than CHUMP, without needing as much information as SLEIGH. The output of ISDL will also be the same as CHUMP, a single disassembled string, not formatted information like SLEIGH.

3.3.3 Related break-x

The first of the break-xs is break-save which is closely related to other time-travel debuggers. The aforementioned WinDbg is also an example of a debugger that supports (T)ime-(T)ravel (D)ebugging. This is where the program can be stepped-back to restore the program state to before some instruction executed. GDB also supports time-travel debugging, whereas LLDB currently does not. Break-save is a less advanced version, as you cannot restore the full state of the program like in real TTDs. Instead users can view the value of variables at points of execution along with the control flow steps in each of the saved stack frames. This form of TTD is at a source level, tracking changes to variables and control flow such as if statements and function calls. Existing TTDs focus at an assembly level, where the effects of an instruction can be completely reversed, restoring the state.

The second break-x is break-on-cause, compared with break-save which saves the past state, break-on-cause is used to break at the earliest point in execution where the debugger can guarantee that the value of a variable will later evaluate to some specified value. A similar process to this is symbolic execution [20] which is used to determine what inputs or variable constraints lead to certain parts of the program executing. Code 1 shows an example function where symbolic execution would represent the constraint on `return 1;` executing as (x_sym * 2) == 12, which can then be solved to give the concrete restriction of x_sym= 6.

```c
int f(int x) {
  int y= x * 2;
  if (y == 12) {
    return 1;
  }
  return 0;
}
```

Code 1: Symbolic execution example

Break-on-cause is similar to symbolic execution however its implementation is different. In symbolic execution the symbolic expressions are formed from moving forwards to each exit point exit point (e.g. (x_sym * 2) == 12), and then a solver calculates the restrictions on the symbols. Break-on-cause starts at the exit points and unwinds the expressions, based on the compiler information, until it reaches a value it can check against the target value.

3.4 Linux-x64

3.4.1 ELF

On Linux the executable file format is ELF [21]. This format contains different sections, such as the text section for the program instructions, DWARF sections such as the debug_line for line number information, several string sections that contain debug strings, as well as eh_frame data which is used for frame information. Parsing the file requires first reading the header which contains

information such as the version, entry point, file type, machine etc. [21, sec. 1-4]. The header also contains pointers to two tables, the program header table, and the program section table.

3.4.1.1 Program headers

The program header table lists the different relocatable sections of the ELF and the link between their virtual address and location in the file. These are then placed in physical addresses later by the loader. Table 1 shows some example program headers. The virtual address of the entry point for this program is 0x1040, so the second LOAD segment contains the entry point.

Type	File offset	Virtual addr	RWX flags
Memory size			
PHDR	0x0040	0x0040	R - -
0x02d8	INTERP	0x0318	0x0318
R - -	0x001c	LOAD	0x0000
0x0000	R - -	0x0620	LOAD
0x1000	0x1000	R - E	0x045d

Table 1: Example part of program headers

3.4.1.2 Section headers

The program section table contains information on the aforementioned sections, the main parts are the name, type, and address.

Name	Type	Address
.dynstr	STRTAB	0x0468
.init	PROGBITS	0x1000
.text	PROGBITS	0x1040
.eh_frame	PROGBITS	0x20a0
.data	PROGBITS	0x3000
.debug_info	PROGBITS	0x311d

Table 2: Example part of section headers

3.4.1.3 Runtime and virtual addresses

One important part that is needed throughout Anura is conversion between the virtual addresses that are used throughout the DWARF and ELF information, and their runtime address that they have been mapped to. Conversion from virtual to runtime is done in several steps. Take the aforementioned program headers that are stored in the ELF data, and find the segment that contains the virtual address, based on the virtual address and memory size attributes. The important parts of this segment are its base virtual address and its offset.

Next we need to look at Linux's /proc/<pid>/maps file, where the pid is that of the target processes. This file contains a list of the processes currently mapped memory regions in the form of address, perms, offset, dev, inode, pathname. The important parts here are the address, which is a runtime address range, and the offset which is the offset into the file. This offset matches with the offset in the program segment data. We can iterate through this file at the launch of the target process and then when converting we search for the offset we got from the program segment earlier.

Now we have a virtual address to translate, the runtime base of the loaded segment, and the program segment's base virtual address. The target virtual address minus the base virtual address

gives the offset into the segment, which is then added to the runtime base, to give the runtime address.

```c
runtime_addr virtual_to_runtime_addr(virtual_address v_addr) {
    const ProgSeg* segment= get_segment_enclosing_vaddr(v_addr);
    virtual_address p_offset= segment->p_offset;

    const ProcMap* proc_map= get_procmap_at_offset(p_offset);
    runtime_address runtime_base= proc_map->base;

    return runtime_base + (v_addr - segment->p_vaddr);
}
```

Code 2: Pseudocode for converting between virtual and runtime addresses (assumes no errors)

When mapping from a runtime address to a virtual address a similar process is done, the /proc/<pid>/maps entry that contains the runtime address is found, which contains a base runtime address as well as a base virtual address (offset). It is important to note however that

3.4.2 DWARF

DWARF is the debugging information format for ELF [22]. This information contains different sections. In the debug_info section there are Debug Information Entries (DIEs) which contain information on the compilation units, the sub-programs within them, the variables, types, and lexical blocks [22, sec. 2.1]. DWARF also contains a line number program which stores information on converting the program addresses into source code lines, this is contained in the debug_line section [22, sec. 6.2]. DWARF information also contains a debug_frame section which describes how to restore the registers of the previous frame at an address, this is used to unroll the stack into a trace [22, sec. 6.4], debug_frame is often omitted and instead replaced with eh_frame which is used for exception handling and contains a very similar format to that of debug_frame with changes to the header [23]. There are several sections that are used to store debug strings, for example debug_str, and debug_line_str, which are referenced by other debugging information.

3.4.3 x64

4 Basic debugger

Anura was developed in C99, using gtk4 as it's UI library. C99 was chosen as it exposed much of the low level interfaces needed, for example ptrace. Gtk4 is a multi-platform UI library [24] and provided an extensive set of UI widgets. The following section describes the implementation details of creating a basic debugger, i.e. the basic features of Anura.

4.1 Target abstraction

To help with extensibility Anura abstracts functions that are target specific into a large Target struct. Upon start up of the target process the corresponding start up function for the target architecture-OS combination sets these functions to its internal implementations. For example, when waiting for the process to hit a breakpoint or another debug event in Linux requires calling waitpid [25]. In windows this is done by calling WaitForDebugEvent [26], the abstracted function target_wait therefore exists in the target struct. These are called throughout Anura via the global target variable.

4.2 Launching a target

The next step for a debugger is attaching to the target process. This can be done in two different ways; attaching to an existing process, or launching a new process. Launching a new process is done in several steps; first by forking the thread, the child of this fork will become the new target and

calls ptrace with PTRACE_TRACEME, and the parent of the fork now has the target's process id. This is due to a Linux security module called YAMA [27], this module contains a scope called ptrace_scope which dictates which processes are allowed to trace other processes. For example, the least restrictive is a process can trace another with the same user id, the next level is that it must be a process with an existing relationship for example a parent of a child process, the next is admin-only tracing, and then no tracing allowed. In order to avoid having to change these scopes PTRACE_TRACEME can be used by a tracee to circumvent the first three restrictions, allowing the debugger to trace it. After this a pipe is created to connect the standard output of the target for the GUI to display in its terminal output. Then execvp is called which replaces the current thread's image with that of the target's. At this stage there is now a running target, and this combines with attaching to an existing process.

Attaching to a process requires it's process id but only require's calling ptrace's PTRACE_ATTACH. If the permissions of tracing are correct for the target then it will receive a STOP signal at the earliest point in execution, which can be caught by the debugger.

4.3 Control thread and actions

Anura is split into three different threads, which requires some inter-thread communication. The most important thread is the 'control' thread, this is what launches the target process, waits on the process for signals, and controls it through ptrace commands. The second thread is the TUI thread which is used to read user input and convert it to commands for the control thread. The final thread is the GUI thread, which for Anura must be the main thread as gtk4 requires that it is run in the main thread context [28]. By splitting the control and UI threads the user can still interact with Anura while the control thread blocks on the target process, waiting for signals.

The TUI and GUI threads generate 'actions' that are placed in a blocking queue for the control thread to complete. This is because interaction with the target process usually requires that it is in a stopped state. For example reading/writing register values, or memory values which most breakpoint actions require cannot be done unless the process is stopped. If one of the UI threads was to attempt reading process memory while it was running it would receive a 'no such process' error.

When the control thread receives a signal for the child process, and is not in a state to continue automatically, it will go through the actions in the queue and implement their actions, this could include control flow actions such as step-over/continue/exit, or display actions like display the unwound stack, or breakpoint actions such as adding or removing them from the target. Each of these actions has a resolution enum which dictates if the control thread can either continue reading from the action queue, or should go back to waiting on the process, or if the process is now exited.

4.4 DWARF DIEs

As mentioned DWARF contains a section filled with (D)ebug (I)nformation (E)ntries which contain information on the compilation units, and their subroutines, types, and variables within. This information is decoded and stored as 'virtual' entries. These are used throughout the program, for example virtual variables contain information on their location and virtual type which can be used to save the value. Virtual types todo:

DIEs like a lot of DWARF information are self describing, this is done by having two sections, the abbrev section which contains the different DIE forms, the fields and types of those fields, the second section contains the actual instances of the DIEs.

The abbrev table has a repeat of the following

| CODE | DW_TAG | has_children (| DW_ATTRIBUTE | DW_FORM)* | 0 | 0

The CODE is a unique number to reference this entry. The DW_TAG is a set enum that describes what kind of DIE this is, for example a DW_TAG_variable, or DW_TAG_subprogram. The has_children field is a boolean that will be used when decoding the data. There is then a number of attribute, form pairs for example; DW_AT_name string, DW_AT_decl_line data1, DW_AT_type ref4, DW_AT_location exprloc. The form is important for when decoding the actual data to make sure the correct number of bytes is read and that the data is interpreted correctly. Finally there is an attribute-form pair that are both zero, this marks the end of the entry. These entries are parsed until the CODE is zero, in which case the table is complete.

Once all tables have been parsed, the debug_info that contains the DIE instances can be parsed. This section is hierarchical, as from before the DIEs can have children, the top level DIEs in this section are (C)ompilation (U)nits, of which there is one for every source file after it has been preprocessed. These CUs first contain a header which contain some meta information including the length of the compilation unit, the size of addresses on the target, and an offset to the abbrev table that it uses. This can be used to reference the correct table from parsing debug_abbrev. Following this are a number of DIE instances starting with a CODE to specify which entry in the abbrev table this is an instance of. If this code is 0 then all child nodes at the current depth have been parsed, and if the depth returns to zero then we have finished parsing all the children of the current CU. Otherwise we take the abbrev entry and for every attribute-form pair we read in the data and store it in the DIE.

4.5 DWARF Line number program

Part of setting breakpoints is being able to place them at source-level lines instead of addresses, and have the debugger do the work of converting them. This requires quite a lot of debug information that is generated by the compiler. You need the range of addresses, their file name, line number, and column number. Naively you could store this as a matrix with a new row each time one of these elements change, however the amount of memory required for this would become unfeasible. Luckily lots of information would be repeated for each row. The file name for example, is easy to see, there will be a number of files but writing the name for each is not needed, they would change very infrequently. Line numbers have this on a smaller scale, for most lines there will be a fairly large range of addresses that were generated by it and so in the matrix the address would increase in each row but the rest of the information would remain the same.

The debug_line section is designed to hold this information and does so by creating a byte code that records the changes in addresses, line, and column numbers, as well as if a new statements has been made, file name has changed, etc. One of the unique parts of this encoding scheme is that at the start of the program it describes the structure of the opcodes, they are split into standard, extended, and a special opcode. Special opcodes are the most common opcode as they encode an op increment (which is used to calculate the pc address) and line increment into one byte.

opcode	count
Special op	16620
DW_LNS_set_column	13594
DW_LNS_advance_pc	5383
DW_LNS_const_add_pc	2876
DW_LNE_set_discriminator	2502
DW_LNS_negate_stmt	878
DW_LNS_advance_line	726
DW_LNS_copy	381
DW_LNE_set_address	25
DW_LNE_end_sequence	25
DW_LNS_set_file	5

Table 3: Example opcode counts taken from Anura's debug info

The encoding of the special opcode is based on the header information, and is designed to give the widest range of line and op increments. There are three pieces of header information that affect how special opcodes are evaluated; the opcode_base which is the minimum opcode that is interpreted as a special opcode instead of a standard opcode, the line_base which is the minimum line advance value that can be encoded, and the line_range which is the number of different line advances that can be encoded. Table 4 shows the first four rows of mappings from special opcode value to their op advance and line advance values, based on a header with opcode_base= 13, line_base= −3, and line_range= 12.

———	Line advance											
Op adv.	−3	−2	−1	0	1	2	3	4	5	6	7	8
0	13	14	15	16	17	18	19	20	21	22	23	24
1	25	26	27	28	29	30	31	32	33	34	35	36
2	37	38	39	40	41	42	43	44	45	46	47	48
3	49	50	51	52	53	54	55	56	57	58	59	60

Table 4: Snippet of DWARF [22, Figure. D.34] Example line number special opcode mapping

With this encoding scheme it is not possible to represent an op advance of 2 with a line advance of 10, and so at least one standard opcodes would have to be used which would increase the size of the line information. To optimise this the compilers DWARF information will change these three values based on the target to maximise the range. For example “For a machine where pipeline scheduling never occurs, it is advantageous to trade away the ability to decrease the line register in return for the ability to add larger positive values to the address register” [22, Sec. 6.2.5.1], this would mean increasing the line_base to 0 which would allow line increases of 9, 10, and 11 to be represented if the line_range remained the same, or would increase the op advance values if the line_range was decreased instead.

It is also possible for a compiler to omit some section of the standard opcodes given that they are all contiguous and include the highest value standard opcode. This allows the opcode_base to decrease, increasing the number of special opcodes.

As this information is stored as byte code Anura contains a virtual machine that runs through the line number program of the target. As this produces a large amount of data there are some

techniques used, for example when finding the address of line x the virtual machine is run up to the point that line x is generated, anything after is not evaluated but the state is stored so that when line y, $y > x$, is queried it can continue from the previous point.

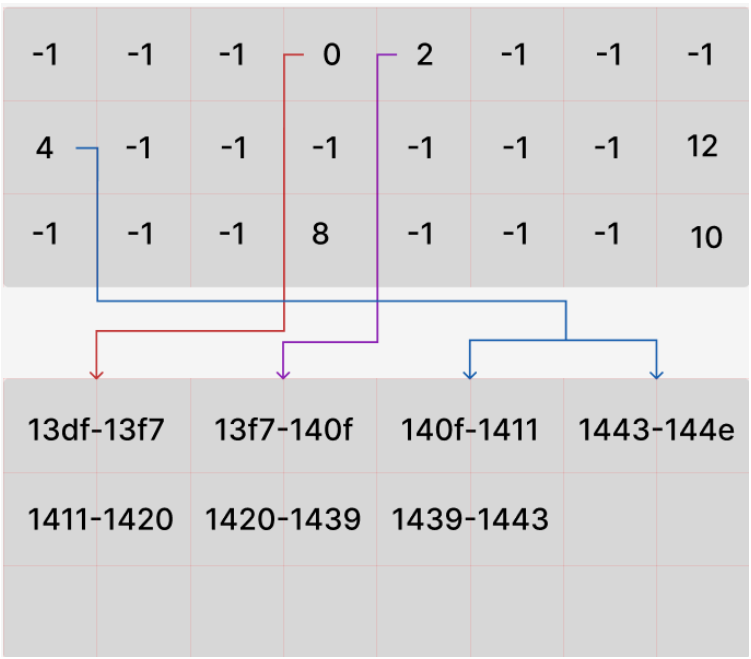

Figure 2: Line number storage for Line->Address

The data is stored in two different formats to allow for quick querying from addresses to lines and from lines to addresses. For lines to addresses, every source line has an entry in a table that is either −1 for no address ranges, or an offset into a ranges list. At the offset is a number of ranges linked to that source line. The table is maintained to make sure the ranges are stored contiguously and in order in a way very similar to a heap. For address to line information the address ranges are stored in a sorted list which can then be binary searched over to find the containing line.

4.6 Frame information

Most debuggers provide some way of looking at the call stack, displaying the current frame and the function it's associated with and then recursively unwinding the stack to find where in the parent it was called from and where in the parent of the parent that was called from. This unwinding should also include restoring the value of the registers in the parent's frame which allows data in that frame to be viewed if it was overwritten and to be restored by the current frame.

A stack frame is an area of memory associated with the function instance, for the Linux x64 abi this contains the return address, the saved frame pointer, and then an area for local variables. It can also contain stack arguments when calling another function. [https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf 3.2.2 The Stack Frame]

Position	Contents	Frame
8n+16(%rbp)	nth stack argument	Previous
	...	
16(%rbp)	1st stack argument	
8(%rbp)	Return address	Current
0(%rbp)	Previous frame pointer	
−8(%rbp)	Local data	
...		
0(%rsp)		

Table 5: X64-Linux Stack Frame [https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf 3.2.2 The Stack Frame]

As part of the x64 abi there are a number of registers that are listed as volatile and non-volatile, meaning they can be overwritten by the callee before use and that they must be preserved by the callee respectively. When unwinding the stack we want to restore the preserved registers as they might be used in a previous frame, or could be needed for the unwinding itself. Hence as part of the debugging information there is data on how to restore registers at different parts in the program. There are two different areas to find this information, firstly there may be a DWARF debug_line information entry, or more commonly an eh_frame entry. eh_frame data is there as part of the exception handling information for unwinding the stack when an exception occurs. They both follow very similar formats as eh_frame is based on the debug_line format.

As mentioned before the start of the frame is marked by the frame base value, this could be in x64 the value of rbp, the base pointer, however the register may be omitted and it may instead be an offset from the stack pointer, rsp, which can change as the function progresses. These are abstracted in DWARF and referred to as the CFA, the canonical frame address. The first goal of the DWARF information is to record how to calculate the value of the CFA at the different addresses. Secondly is storing how to recover the value of the other registers, which often have their value at some offset from the CFA.

4.7 Breakpoints

Breakpoints are places in the execution that the target program should stop and wait for the user to continue. This allows the user to view information and alter data. There are two types of breakpoint implementation; hardware and software.

4.7.1 Hardware breakpoints

In x64 there are 8 hardware registers associated with debugging. Two are reserved being DR4 and DR5. DR7 is the debug control register and controls which breakpoint registers are active, the type and length of the breakpoint, as well as two flags. DR6 is the debug status register which contains information on the last debug exception that was generated, for example a bit for each of the hardware registers to specify if they were hit, as well as a single step flag for if the instruction was single stepped. Debug registers DR0-3 contain the addresses to break on.

Outside of implementation there are

4.7.2 Software breakpoints

Software breakpoints involve placing some instruction that will cause an exception to generate when the processor reaches it. These can be tailored instructions like x64's INT3 [29, Vol. 2A,Sec. 3-467] which generates a breakpoint exception when hit, or by placing some illegal/

invalid instruction that will generate an exception. The debugger receives all signals for the child process first and so can intercept these exceptions and choose how to handle them. Placing a software breakpoint requires storing what the original byte(s) were, I'll refer to this as the shadow. X64's hardware instructions generate the exception before the instruction is executed and so the instruction pointer points to the address, whereas for software breakpoints the INT3 instruction has been executed to generate the exception and so the instruction pointer is one ahead. In order to continue execution for any reason, be it single stepping, or full running, we have to replace the original instruction so that it can be executed. This requires placing the shadow back, single stepping, and then replacing the trap instruction again.

4.8 Stepping

There are two levels of stepping, assembly level and source level. On the assembly level there is only one option, to step over to the next instruction, this is typically done at the cpu level where there is some mechanism that makes the CPU execute one instruction and then generate an exception. In x64 this is done through the TRAP flag, which when set will cause the CPU to generate a TRAP exception after each instruction is executed [30, Vol 1, sec. 3-17], this exception can then be captured by the debugger.

At the source level there are three different steps, step-over, step-into, and step-out. Step-over means go to the next line in the current function, skipping over any function calls in the current line. Step-into means go into the next function called by this line. Step-out means go to the return address of the current function.

```c
int f() {
  // step-into destination
  return 1;
}

int g() {
  int a= 1;
  a += f();
  // Starting point
  return a;
  // step-over destination
}

int main() {
  int x= g();
  return x;
  // step-out destination
}
```

Code 3: Stepping destinations

Code 3 shows the destinations of each different type of stepping given that execution is currently on line 7.

4.8.1 Step-into

Step-into is fairly simple in implementation, we have some starting source line, and we want to break execution when this line changes. There are two reasons this line could change, either we enter a new calling function or we go to the next line in the current function. In either case we want to break, as we don't know which function the program may be calling next we cannot just set a break point and run to it, instead we can take advantage of the fact that the amount of instructions

for a line are typically low, and so we will single step the program until the address the target is at maps to a different source line.

4.8.2 Step-out

Step-out is also fairly simple, we want to get out of the current function, returning to the next instruction after it was called. This requires finding the return address for the current function instance. This is stored in the aforementioned frame information, as, in what was a revelation for me, the return address of a frame is the restored instruction pointer of the previous frame. So, from the cie we get the return address register value, which is DWARF R16 for x64, and then restore it's previous frame value, place a break point at that location and run until it is hit.

4.8.3 Step-over

Step-over is the most complex of the three, unlike step-into we cannot only single step though until we reach the next line of the current function, because there may be function calls in the current line that have thousands of instructions, single stepping all of them would slow the program to a halt. Instead we consider our current line from two references, it's line number, and it's CFA. The CFA is the canonical frame address, and is DWARFs abstraction of frame base. By having these two references we can represent our requirement of next line in the current function as the line numbers being different, but the CFA being the same. There are three other situations to handle, we could be on the same line and the same CFA, in this case we single step. In other cases the CFA value may have changed, this could be for two reasons. Firstly we could have entered a new function causing the CFA to reduce, in this case we've entered a new function that we want to entirely skip, luckily this just requires stepping-out of the function, and so we find the return address and set a breakpoint. In the other situation the CFA has grown, this is usually because the current function has ended and in this situation all we can do is break.

4.9 UI

4.10 Register values

One very standard part of a debugger is some way to view the contents of the registers, both the general purpose and more special case ones. In the Linux-x64 environment getting general purpose registers is very easy, there is a ptrace action called GETREGS [31] which will copy into a struct which is exposed by <sys/user.h>. There are however, a number of non-general purpose registers in x64, the vector registers, and the floating point registers. Access to the data of these is done through ptrace's GETREGSET with an argument of NT_X86_XSTATE, however there is no exposed struct that will parse and hold this data. Instead this is a raw copy of x64′s XState which we must decode.

One part that makes decoding more complicated is that the vector registers were added in a series of CPU extensions, and so not all the registers exist on each machine, for example on the machine most of Anura was developed on SSE, AVX, and AVX2 are supported, but AVX512 is not.

The first step is to get the size of the XSAVE area, this is done by querying the cpuid instruction. The CPUID instruct has two arguments that are implicitly EAX and ECX. EAX is referred to as the leaf and it describes the main category of the function, ECX is the sub-leaf and can describe a different instruction within that category. With a leaf of 0xD and sub-leaf of 1, the size of the XSAVE area is placed in EBX. We then allocate a buffer for this and pass it to a ptrace call for GETREGSET on NT_X86_XSTATE.

As with lots of the x64 instruction set, the XSAVE area is comprised of both legacy and modern formats. The first 512 bytes of the region comprise the legacy region and contain information on the floating point and SSE registers. For decoding registers we can ignore the first 16 bytes, there are then 8 instances of a 10 byte STx register followed by 6 reserved bytes. These are the 8 STx floating

point registers that are also aliased to the 8 Multi-Media Extension registers. Following these are 16 instances of the 16 byte XMMx registers which are part of SSE, a 128-bit vector extension. The remaining bytes of this legacy region are not used.

Following this is the 64 byte XSave header. The first 8 bytes comprise the XStateBV which is state component information, and the second 8 bytes comprise the XCompBV, which contains both a single bit of used to decide the format of the next section, between standard and extended. This follows a 63 bit region which is used to decide which state components are active in the extended format. The remaining bytes of the header are currently unused.

```c
typedef struct XSaveBV {
  uint64_t x87_state: 1;
  uint64_t sse_state: 1;
  uint64_t avx_state: 1;
  uint64_t mpx_bnd_reg_state: 1;
  uint64_t mpx_bnd_csr_state: 1;
  uint64_t avx512_opmask_state: 1;
  uint64_t avx512_zmm_hi256_state: 1; // High 256 bits of ZMM0-15
  uint64_t avx512_hi16_zmm_state: 1;  // ZMM16-31
  ...
}
typedef struct XCompBV {
  uint64_t format: 63;
  uint64_t is_ext_format: 1;
} XCompBV;
```

Code 4:

Each of the components stored in the next region are called state components. The format of the region is dynamic, in that it can change dependant on which state components are available and how many bytes they require. This information is available through another cpuid query using the same leaf of 0xD, but a sub-leaf value equal to the state component value listed in Code 5, because x87 and SSE are stored in the legacy region they are excluded. This information includes an offset, a size, and a boolean for alignment.

```c
typedef enum STATE_COMPONENT {
    SC_X87=0,
    SC_SSE=1,
    SC_AVX=2,
    SC_BNDREGS=3,
    SC_BNDCSR=4,
    SC_AVX512_OPMASK=5,
    SC_AVX512_HI256=6,
    SC_AVX512_HI16=7,
} STATE_COMPONENT;
```

Code 5:

All of this data then allows the parsing of the main body, which comes in two different formats; standard and compact. The form used is based on the XCompBV bit in the header. In the standard format the state components exist at the offset specified by their cpuid query where the base of the offset is after the header.

Decoding the extended format goes as follows; iterate through the state components, if the bit in the format field of XCompBV is not set then move to next component. If it is set and it is the first

component set then its offset is 0 from the end of the header. From this point we track what the last component that is active is. All subsequent active components will have their offset at the last offset plus the size of the last, and if the align bit of the component is set it will be aligned to the nearest 64 byte boundary.

Regardless of the format used the offset and size information can then be used to place each component into a representative struct for it's value. It is then possible to display the contents of these registers, typically as a sequence of bytes for the vector registers, and as floats for the floating point registers.

4.11 Variable instancing

Throughout parts of the debugger it is useful to capture the value of a variable to be displayed later. This is done mainly through the virtual data structures,

4.12 Stack frame saving

5 ISDL

5.1 Motivation

As part of Anura's goal in being extensible ISDL is a language designed to help with generating disassemblers for each of the supported target architectures. The aim of this is to reduce the cost of adding a large range of targets. ISDL was designed to have a source language that is similar to the data sheets found in instruction set manuals, in order to reduce the amount of translation required of the developer. The output of the disassemblers is a single string of the disassembly, and the size of the instruction is reversible from the amount of the stream that was consumed. For Anura this is enough information to display the disassembly with it's virtual address.

5.2 Design

ISDL was designed with most inspiration from CHUMP [15], which is a series of what ISDL refers to as left-right rules, which describe what input maps to which output. These rules make up the overwhelming majority of ISDL, and are found mainly in alias statements which are repeatable lists of rules. The following section will describe the final design ISDLs different statements.

5.3 `STRUCTURE` statement

The structure statement describes the top level structure of the instruction set and is allowed to use some special operators.

X64's structure statement is shown in Code 6. The special operators are the same as in a regex, where '*' allows for any number of instances of the identifier, including zero. '?' allows for one or no instances.

ISDL

```
1 STRUCTURE lprefix* prefix? op = op
```

Code 6: x64 structure statement

This is needed as in x64 there can be a number of legacy prefix bytes, followed by at most one modern prefix byte, followed by the actual operation. The output of the structure statement is the output of the disassembly.

5.4 `ALIAS` statement

The alias statement is the main part of the language, there are two types of aliases, the first is the standard alias which contains a list of rules. When the alias is in the left side of another rule it goes

through its rules in sequence until one matches. These rules can be left rules, left-right rules, if rules, or with rules.

5.4.1 Left rules

Left rules are a list of other rules that the alias should attempt to parse one by one, until one succeeds. An example of this is the alias for operations.

ISDL

```
ALIAS op= {
  PUSH
  SUB
  HALT
  CALL
}
```

Code 7: operation alias

5.4.2 Left-right rules

Left right rules, as mentioned, are the main set of rules. They contain a list of rules on the left that if all parse successfully from the input will result in the value of the alias evaluating to that of the right. For example Code 8 shows some left-right rules in the ADD operation. Reading in `1000 0011` followed by whatever the `ModRM_5` rule is and then the `imm8` rule will result in the string `"ADD {ModRM_0}, {imm8}"` being the value of ADD. Once an alias has a value that can be used in the right side, for example if a left-right rule was '`1000 ADD = "REPNE {ADD}"`' and `ADD` parsed to `"ADD rax, rbx"` then the output would be `"REPNE ADD rax, rbx"`.

ISDL

```
ALIAS ADD= {
  0000 010 ~ow immM32= "ADD {regA}, {immM32}"
  1000 000 ~ow ModRM_0 immM32 = "ADD {ModRM_0}, {immM32}"
  1000 0011 ModRM_0 imm8 = "ADD {ModRM_0}, {imm8}"
  0000 00 ms ~ow ModRM = "ADD {ModRMS}"
}
```

Code 8: Add left-right rules

5.5 RULE statements

Rule statements attach to alias statements, they are designed for when an alias statement has multiple possible right rules on the same left rule. This comes up when decoding registers. In x64 the register rax can also be eax, ax, al, and ah, based on the operand size.

ISDL

```
ALIAS reg 4 BITS = {
  0000= "AX", "AL", "AL", "EAX", "RAX"
  ...
}
```

Code 9: Register rule

Rule statements choose which of the right rules to use as the value of the alias when it's used in other right rules. Code 10 shows the rule for regO and regT which are two instances of register. They consist of a series of CHOOSE rules which choose a certain index based on some condition, typically based on flags, which are discussed next.

ISDL

```
RULE RIGHT ON regO, regT {
  CHOOSE 0 if opmode == 16bit
  CHOOSE 1 if opmode == 8bit && REX.w != 1
  CHOOSE 2 if opmode == 8bit
  CHOOSE 3 if opmode == 32bit
  CHOOSE 4 if opmode == 64bit
}
```

Code 10: Rule for registers

5.6 FLAG statements

Flag statements are very simple, they create an enum with certain values and create an instance of the enum with a default value. This is useful where-ever there is a choice between a set of known modes, for example in x64 the operand size, and address size both have their own mode flag, which is used by other parts to change the output.

ISDL

```
FLAG opmode= 64bit | 32bit | 16bit | 8bit default 32bit
```

Code 11: Operand size flag

5.7 CALCULATE statements

Calculate statements are to flags as rule statements are to aliases. They provided a way to change the value of the flag based on some conditions. When a flag is first used in any way it's calculate statement is called to evaluate it. This allows parts of the instruction to be decoded to provide enough information to discern values such as the operand size, which is based on prefix bytes and a bit, the ow bit, found in most instructions.

ISDL

```
CALCULATE opmode= {
  if ow then 8bit
  if REX.w then 64bit
  if lp3? then 16bit
  default_opmode
}
```

Code 12: Opmode calculate statement

5.8 DATA statements

Data statements were initially designed as special types of aliases but were moved to their own statement. They represent a part of an instruction that stores data, from an immediate operand, to the register encoding bits, to a prefix byte's structure. They come in two forms, fielded and non-fielded. Code 13 shows three data statements, the first which is storage of a 16 bit immediate value, and the second and third of which are structured prefix bytes. Fielded statements allow for constants and named bits, these are 1 bit by default but can be specified as larger. Default values for these fields can also be given.

ISDL

```
DATA imm16 2 BYTES
DATA REX= {
  0100 .w=0 .r .x .b
}
DATA VEX= {
  1100 0100 .R .X .B .m(5) .W .v(4) .L .pp(2)
  1100 0101 .R .v(4) .L .pp(2)
}
```

Code 13: Data statements; fielded and non-fielded examples

5.9 WITH and VAR statements

When finishing the original version of ISDL it became clear that one assumption was a problem, that being the calculation of operand size. For a lot of instructions the opmode's size i.e. if it is 8/16/32/64 bits uses Code 14, where 32 bits is the default size.

ISDL

```
CALCULATE opmode= {
  if ow then 8bit
  if REX.w then 64bit
  if lp3? then 16bit
  32bit
}
```

Code 14: Operand size calculation

This is not true for all instructions however, and in 64 bit long mode some operations like PUSH have certain opcodes that have a 64 bit default operand size. This is a problem because in the original language design opmode is used throughout the language to decide if a register should decode as rax or eax, or if a 16 bit immediate should be read or a 32 bit, and choosing which pointer string should be used e.g. DWORD PTR or QWORD PTR. One fix for this would be to create two version of these registers, ModRM byte, immediates etc. however this would be tantamount to writing the language specification twice. The language therefore needed some way of changing data but only for the current instruction line.

Two statements were introduced to fix this. Firstly is the VAR or variable statement. These are extensions to the FLAG statements with a structure of

ISDL

```
VAR <var_name> OF <flag_name> = <flag_value>
```

This allowed variables to be used in place of normal flags, which includes the CALCULATE statement.

ISDL

```
CALCULATE opmode= {
  if ow then 8bit
  if REX.w then 64bit
  if lp3? then 16bit
  default_opmode
}
```

The second part is to allow instructions to change this variable when decoding, this is done through WITH statements.

ISDL

```
WITH <var_name> = <flag_value> {
  <rule>
  <rule>
```

```
}
```
ISDL
```
ALIAS PUSH= {
  WITH default_opmode = 64bit {
    0xFF ModRM_6 = "PUSH {ModRM_6}"
    0101 0 regop = "PUSH {regop}"
  }
  0110 10 ow 0 immM32 = "PUSH {immM32}"
  0x0F 0xA0 = "PUSH FS"
  0x0F 0xA8 = "PUSH GS"
}
```

This means that operators that need to override the operand size without affecting other parts of the decoding, if this part of the decoding fails then the default_opmode variable is restored.

5.10 MAP statement

It is very common for x64 to take bits of information from different sections of the instruction and combine them to represent something. This mainly occurs with the registers which can be extended by bits the REX prefix byte to form a 4 bit mapping to registers. The majority of the bits for registers are combined with the opcode byte or in the ModRM byte.

For example take a ModRM byte of 11 101 000, and REX byte of 0100 .w=0 .r=1 .x=0 .b=0. The register encoded in the ModRM's register part is made up of both the 3 bits in the ModRM byte and the 1 bit REX.r field. The full 4 bit encoding is 1101, which encodes R13.

The description language therefore requires some way of combining bits of information. This could be done by creating an alias as follows

ISDL
```
ALIAS reg_with_r= {
   if (REX.r) {
      000= "R8W", "R8B", "R8B", "R8D", "R8"
      ...
      101= "R13W", "R13B", "R13B", "R13D", "R13"
   }
   000= "AX", "AL", "AL", "EAX", "RAX"
   101= "BP", "CH", "CH", "EBP", "RBP"
}

```

This works however, consider if we need to combine the register bits with a different bit. For example REX.b is combined with the 3 bits in the SIB byte. There would then be another alias with all the registers written out again. In fact we need one for the REX's r,b, and x fields.

This lead to the introduction of the MAP statement. The general structure of which is

ISDL
```
MAP <dst_alias> <input0> <input1> ...
```

This allows instances of different registers that combine the needed bit information and pass it into another alias as the input stream.

ISDL
```

DATA regx 3 BITS
ALIAS regr 3 BITS = {
    regx = MAP reg0 REX.r regx
```

```
5 }
6 ALIAS regop 3 bits = {
7     regx = MAP regO REX.b regx
8 }
```

The above statements mean read in regx which is a 3 bit data field and then create an input stream with the bit REX.b followed by the 3 bits of regx and get the output of regO with that input stream.

regO is an instance of reg which is simply a mapping of 4 bit inputs into the register strings.

ISDL

```
1
2 ALIAS reg 4 BITS = {
3   0000= "AX", "AL", "AL", "EAX", "RAX"
4   ...
5   1101= "R13W", "R13B", "R13B", "R13D", "R13"
6 }
7 ALIAS regO= reg
8
```

5.11 ISDL improvements

5.11.1 Optimisation

Currently in ISDL there are no optimisations, one impactful one would be improving how the correct instruction is selected. In the unoptimized version this is done sequentially with the opcode tested each time. To improve this would require having a sorted list of the first byte and being able to binary search through based on the opcode. There are two problems to this, one is that not all opcodes are an entire byte which I'll discuss later, and second is that the opcode value is hidden behind aliases as seen in the operator alias [x]. To fix this ISDL could propagate up constants at the start of a rule, if done until all constants are completely propagated the operation alias would look like

ISDL

```
1 ALIAS op= {
2   0xFF PUSH_000 = PUSH_000
3   0x83 SUB_000 = SUB_000
4   0xF4 HALT_000 = HALT_000
5   0xE8 CALL_000 = CALL_000
6   0xFF CALL_001 = CALL_001
7 }
```

This would then allow sorting on the byte and jumping to the correct start without having to go linearly through. As alluded to before not all opcodes start with a unique one byte opcode, and with the way ISDL is designed some contain information e.g. the ow bit found in a lot of instructions decides if the operand is 8 bit, and instead of writing two instruction lines with virtually the same information it is DATA that is used when calculating the opmode. This means that instructions often appear in the form `1100 011 ~ow` or `1011 ~ow regop`. Instead of requiring the constant to be an entire byte it would instead take all constant bits and then create an if-else binary tree with the if being the next bit is a 1 and the else a 0.

ISDL

```
1 ALIAS op= {
2   00001111 10100000 PUSH_003 = PUSH_003
3   00001111 10101000 PUSH_004 = PUSH_004
4   0010110 SUB_001 = SUB_001
```

```
  01010 PUSH_001 = PUSH_001
  011010 PUSH_002 = PUSH_002
  1000000 SUB_002 = SUB_002
  10000011 SUB_000 = SUB_000
  11101000 CALL_000 = CALL_000
  11110100 HALT_000 = HALT_000
  11111111 PUSH_000 = PUSH_000
  11111111 CALL_001 = CALL_001
}
```

As you can see from this small snippet of instructions we already eliminate around half of the instructions each time we read a new bit.

5.11.2 Design

ISDL is extremely generic, it is simply a way of converting binary into formatted string, this makes it quite powerful and potentially useful in other areas. However, it does also mean that it is not as powerful as some disassemblers, mainly in regards to formatted information on the instruction. ISDL doesn't have any way of outputting the opcode of the instruction or the operand(s), it cannot show the conditional code used outside of the instruction mnemonic. It is possible to find the instruction size but only given the fact that the stream has moved a certain number of bytes from the start and end of disassembling. This is an area that ISDL can improve in, either through becoming less generic with more rigid types like an instruction type. Alternatively ISDL could remain generic and have more support for describing how to convert what is read into internal structures that can then be displayed by another program.

5.12 Issues when implementing x64

5.12.1 Data size vs Address size

In x64 there are two different sizes that get encoded, the first is operand size, these are for immediates and registers. There is also the address size this is for registers that are being used as . The default operand size is 32 bits whereas the default address size is 64 bits. As stated before we don't want to have to write out the entire register bank again with a new rule set, so ISDL allows aliases to keep their multiple value list until an alias with rules is used, at which point the actual output is selected.

We can then create a new alias for registers being used in a memory context. Giving them a new right rule

ISDL

```
ALIAS regM= reg
```

ISDL

```
RULE RIGHT ON regM {
  CHOOSE 0 if addrmode == 16bit
  CHOOSE 1 if 0
  CHOOSE 2 if 0
  CHOOSE 3 if addrmode == 32bit
  CHOOSE 4 if addrmode == 64bit
}
```

This allows for decoding the following `MOV DWORD PTR [RBP + 0xfc], 0x1` where the operand size is 32 bits as shown by the `DWORD PTR` but the register being used as the base is rbp, the 64 bit variant of the stack base register.

5.12.2 Reading data

There are two parts of reading data that required special consideration in the language, the first is that in x64 sometimes an instruction bit can mean the inverse of how it's usually used. For example the previously mentioned ow bit has the meaning `The operand is 8 bit if ow is set` in some instructions and `The operand is 8 bit if ow is not set` in others, but the ow bit is used the same when calculating the opmode.

Bit pattern	Meaning	ow bit info
0110 1000	PUSH a 16 or 32 bit operand	7th bit
0110 1010	PUSH an 8 bit operand	7th bit, set for 8 bit mode
0010 1100	SUBtract an 8 bit operand	8th bit, not set for 8 bit mode
0010 1101	SUBtract a 16 or 32 bit operand	8th bit

Table 6: ow bit differences in instructions

Reading data inverted is done via the ~ operator, which is placed before a data variable in an alias statement. The second consideration is endianness, specifically when reading instruction data. In x64 data is stored in little endian [30, Vol. 1, Sec. 1-6] which means that the lowest order byte is stored at the lowest address in memory. Consider the data `0x12 0x34 0x45 0x67` in an instruction, the actual value of this is 0x67453412. ISDL contains a `META` statement in which the endianness can be specified, then when reading multi-byte data the byte is placed in the correct location by shifting.

5.12.3 RIP-relative values

Within x64 there are several instances of instruction-pointer relative values, for example `MOV rax, [rip + 0x90]` this instruction when placed at address 0x1000 can actually be disassembled to `MOV rax, [0x1097]`. This is because rip-relative means relative to the start of the next instruction, and the instruction is 7 bytes long, plus the virtual address, plus the offset, to give 0x1097. Initial versions of ISDL could not disassemble to this level, as it did not know the value of the instruction pointer when disassembling. A conceptualised, but not planned, feature for ISDL was the ability to give values to the DATA statements as arguments to the disassembly. This is what led to the current implementation where the disassembly function takes in a rip argument, that then sets the value of the rip data statement. This can then use ISDL's expression evaluator to calculate the actual address. This current implementation is a small part of what ISDL could support in the future, with a more robust argument system.

5.13 Testing

When testing ISDL the output was compared to that of objdump's disassembly on example programs. Testing revealed several issues throughout the process, one of which was the inability to display data as signed, and therefore negative, in expressions that required negative offsets. This was due to the fact that the data statement's values were stored as unsigned values and then evaluated to strings as unsigned values. A temporary fix for this was to sign the stored data, and to evaluate the string in different ways based on if it was positive or negative. This fix however should be developed into a much more robust system to allow for purely unsigned data, this is discussed in the coming evaluation.

The initial set of implemented x64 instructions was made purely off of the example programs. This encompassed the most common instructions, and allows for a wide range of basic programs to be disassembled.

5.14 ISDL evaluation

ISDL currently has the capability to represent the overwhelming majority of instructions within x64. There are currently over 450 different 1-2 byte opcodes, which does not include differentiating parts such as ModRM_x bytes, this makes testing and implementing each unfeasible for this project. Most, however, follow patterns of operand format, operand size rules, address size rules, etc. which can be implemented and re-used in ISDL.

As mentioned, one current flaw of ISDLs design is the lack of distinction between signed and unsigned data. The data statement has no attribution of sign which means that a default had to be chosen, which was signed. As future development and improvement to ISDL, a signed and unsigned keyword specifier could be added to data statements which then treat the string evaluation differently dependant.

One further development to ISDL that would help both in terms of

6 Disassembly in debuggers

6.1 Data in code

In some scenarios it may be the case that the compiler places data in the same area as the generated code. For example if the generated jump table for a switch statement is placed in the text section then when we disassemble we will eventually reach what is actually data and start decoding, which will produce garbage instructions that might extend over the real instructions that come after it, meaning all instructions after are incorrectly decoded. This is uncommon in modern compiler output as data like jump tables are more likely to be placed in read only sections of the binary, however it is interesting to consider how to solve this.

6.2 Linear and Flow Disassembly

There are two main types of disassemblers; Linear disassemblers, and Flow disassemblers. Linear disassemblers start at some address and decode instructions serially, one after the other. Flow disassemblers take into account the control flow of the program, if a decoded instruction alters the control flow unconditionally then the disassembler will simply decode at the destination e.g. JMP 0x1100. If the instruction conditionally alters the flow then the disassembler will place the destination on a stack to be later disassembled and continue down the current flow.

6.3 Using DWARF information

We do however have access to more information, that is, we have a mapping of the source line numbers to their instruction address. This means we have known locations that are the start of instructions. In this case if a line decodes past it's actual address we can decode up to it's end point. This won't affect the next known instruction as we know the start pc value of the next line.

	0x1100	`mov rbx, QWORD PTR [switch_data + rdi*8]`	48 8b 1c fd 08 11 00 00
	0x1108	`jmp jmp_label`	e9 25 00 00 00
`switch_data:`	0x110D	`.long 0xe8010203`	e8 01 02 03
	0x1115	`.long 0x04e80607`	04 e8 06 07
	0x111D	`.long 0x0809e811`	08 09 e8 11
	0x1125	`.long 0x121314e8`	12 13 14 e8
`jmp_label:`	0x112D	`mov rax, rbx`	48 89 d8
	0x1135	`ret`	c3

Table 7: Example assembly with data in code

Table 7 shows an example program where there is data in the same section as the code. For a linear disassembler it would decode the move and jump instructions and then continue into the data where it would decode the byte e8, which is the call instruction and then read 4 bytes for the location, giving call 0x4030206 (the displacement of e8 is relative to the next instruction so after the 5 bytes of this instruction hence the 06 over 01 [29, Vol. 2A, sec. 3-121]). This would then leave the stream looking at the next byte which is another e8. and the process repeats until we reach the last byte of data which is another e8, except this time there is no more data to read as the instruction's displacement and so it reads the bytes of the move and return instruction decoding to call 0xc3d8894d.

```nasm
1 mov rbx, QWORD PTR [0x110D + rdi*8]
2 jmp 0x112D
3 call 0x4030206
4 call 0x9080710
5 call 0x14131220
6 call 0xc3d8895c
```

Code 15: Disassembly based on linear disassembler

For a flow disassembler this is not a concern in this instance as it would follow the jmp control flow to decode at 0x112D skipping all the data. A case where flow disassemblers fail is if the control flow has been maliciously crafted to pretend that it might go into the data when in reality it always skips it. An example of this is shown in Code 16

```nasm
1 xor rax, rax
2 cmp rax, 0
3 jz label
```

Code 16: Malicious assembly to make flow disassembler disassemble data

Although the DWARF information wouldn't be susceptible to this kind of control flow it would require the scenario in which anti-disassembly has been used while still leaving debugging information available, which is not likely. It is also the case that unlike a flow disassembler using the DWARF info would still decode the data as instructions it's only the case that it can recreate the actual instructions.

```c
1 int main(int argc) {
2   long a;
3   switch {
4     case 0: a= 0xe8010203; break;
5     case 1: a= 0x04e80607; break;
6     case 2: a= 0x0809e811; break;
7     case 3: a= 0x121314e8; break;
8   }
9   return a;
10 }
```

Code 17: Example C source program for generated assembly

Line number	Start address	End address (Exclusive)
3	0x1100	0x112D
9	0x112D	0x1136

Table 8: Line number mappings for Code 17

With these mappings we can have the disassembler start decoding at 0x1100 until it reaches 0x112D at which point we start anew from the next source line entry which is 0x112D. The result of which is shown below. We can mark a decoded line as an error or invalid based on if it decodes outside the end address of the line. This will only work on the last instruction that is actually data only if it requires more bytes than are left in the range.

```nasm
mov rbx, QWORD PTR [0x110D + rdi*8]
jmp 0x112D
call 0x4030206
call 0x9080710
call 0x14131220
call 0xc3d8895c # Marked: overflows by 4 bytes
mov rax, rbx
ret
```

Code 18: Disassembly based on DWARF information

Anura implements a slight hybrid of this, it gets a list of all the virtual subroutines in the current CU, it then disassembles as much of the subroutine as it can, when it fails it finds the next source line and fills the view with '???' until that line's address, then it continues disassembling from that point. This is used to prevent unknown/invalid instructions from breaking the disassembly, as not all instructions are currently implemented in ISDL. Table 9 shows the output of Anura when disassembling a subroutine with two instructions that have not been implemented in ISDL, namely the sete and movzx instructions, in this case even if the movzx instruction had been implemented it would not have been decoded because it is not the start of a source line, leave and ret are displayed as they are associated with the last line of the subroutine.

Address	Decoded instruction	Bytes
0x12ee	CALL 0x137e	e8 8b 00 00 00
0x12f3	TEST RAX, RAX	48 85 c0
0x12f6	???	0f
0x12f7	???	94
0x12f8	???	c0
0x12f9	???	0f
0x12fa	???	b6
0x12fb	???	c0
0x12fc	LEAVE	c9
0x12fd	RET – sub parse_y END	c3

Table 9: Anura's disassembly output: Skipping unknown instructions

7 Break-x

There are two additional forms of breakpoints in Anura, break-save and break-on-cause. They are related in the base features they use, as break-save was originally designed when writing an alternatives section for break-on-cause, before implementing it fully. They are designed to help understand the data flow of a program, and the reason that a variable has reached a certain value.

7.1 Motivation

A common format for the parsers is to contain code similar to Code 19, where there is a loop parsing top level statements that breaks early if there is an error in any of these. During the development of

a recent parser I encountered an error in a statement that I hadn't yet put an error message for, which forced me to put a break on the top level function and continue through and count the number of successful and then rerun and stop before the erroring one. There are several solutions that could make this process easier; Placing a breakpoint on the if statement of the main loop and then inspecting the value of calling current would show the last token in the stream. Alternatively, placing a breakpoint on the parse_top function and then counting iterations until failure and restarting and doing one less would also help, writing code free of bugs would help significantly. However, these solutions require much more work from the developer than the debugger.

```c
int parse() {
  ...
  while (stream_has_token()) {
    const int res= parse_top();
    if (res == FAIL) return res;
  }
  return SUCCESS;
}
```

```c
int parse_top() {
  Token* c= current();
  switch (c->type) {
    case TYPE_A: return parse_a();
    case TYPE_B: return parse_b();
    case TYPE_C: return parse_c();
  }
  return FAIL;
}
```

Code 19: Common parsing function overview

This is the base motivation behind break-save and break-on-cause. Break-on-cause was the first developed as it comes from the most helpful outcome the debugger could produce in the parsing situation; being able to break on the value that causes the early failure, which would be some return FAIL down the call stack. Break-save is about saving all the lost function/stack information, that comes from the FAIL being propagated back up and stack frames being removed.

8 Break save

As mentioned, break-save is a lite form of time-travel debugging, where the stack frames from a point of execution are saved and visible to the user. It displays control flow and data flow timelines, to help better understand the execution of the program. Compiler information creates control flow and data flow points with listed reasons at addresses throughout the program. The debugger can then break at these locations and mark control flow points, or save data for data points, be that variables, struct member, or return values. Code 20 shows an example program and the different control and data flow points, along with their reasons, comments are shown below each marked line.

```c
int main() {
  while (current()) {
    // Control flow: while loop conditional
    int res= parse_top_level();
    // Control/Data flow: loop body and res value overwritten
    if (res != SUCCESS) return res;
    // Control flow: if conditional and if resolution
  }
}

int parse_top_level() {
  Token* c= consume();
  // Data flow: c is given a value
  switch (c->type) {
  // Control flow: decision point (same as start of if statement)
    case TYPE_A: return parse_a();
    case TYPE_B: return parse_b();
    // Control flow: possible resolution points
  }
}
// Control flow: possible resolution point
```

Code 20: Data and control flow information for function
(source line level only)

8.1 Compiler generated information

Break-on-save requires the compiler to generate some extra information to work. This is namely; the control flow and data flow points for every function, as well as the function's start and end addresses. The points can be stored contiguously without any kind of sorting.

Both types of points have the same three pieces of information, being a enum type, a virtual address, and an enum reason. Data flow points additionally have a die offset and conditionally a location. The die offset is used to reference the DIE of a variable/parameter/struct member which is then used to capture the value later on. For most data flow points the location of the data can be found through this DIE, however for DF_REASON_MEMBER_ASSIGN the location must be given, for example

```c
t->data->storage.idx= f()
```

In this code break-save needs the location of idx so that it can save this new value at this point. With the existing DIE information it is possible to calculate this offset as t has a type that contains the data member which has an offset which contains a storage member at some offset which contains an idx member at an offset, however using this would require storing the fact that we are starting with t and then dereferencing and member accessing to data, which then dereferences and member accesses etc. Instead break-save information can be stored using a DWARF expressions which combines all of this information into calculating the location of idx.

8.2 Control/Data flow reasons

There are several control flow reasons currently implemented; pre/post function call, pre-conditional, conditional resolution, and frame start/end. There are four different data flow reasons; variable assignment, return value assignment, member assignment, and argument assignment. Most of these are simply when an actual value is assigned to, however argument assignment is special as it should appear after a function call to save the new value of pass-by-reference arguments. This is

because otherwise the caller function frame doesn't know that the values have changed and therefore won't have saved the value.

8.3 Implementation

The first part of a break-save implementation is the check for the condition, this is simply a conditional breakpoint which is found in most debuggers. So the first placed breakpoint has a callback to check if the local variable res is equal to −1, and if so it will break the program and display the information. Next break-save iterates through the subroutines of the program and places breakpoints on their prologue and epilogue. When the prologue is hit a new frame is allocated and the control and data points of the subroutine are placed as breakpoints if they do not already exist. Frames store their child frames in an array, and when a frame ends it's parent becomes the current frame again.

Most control flow point hits are simple, they are added to the current frame to be displayed in order later. However, CF_REASON_FRAME_START points are special, as they save the parameters of the function to the frame. This is similar to all data points which will instance the value, for example instance the variable for a DF_REASON_VAR_ASSIGN.

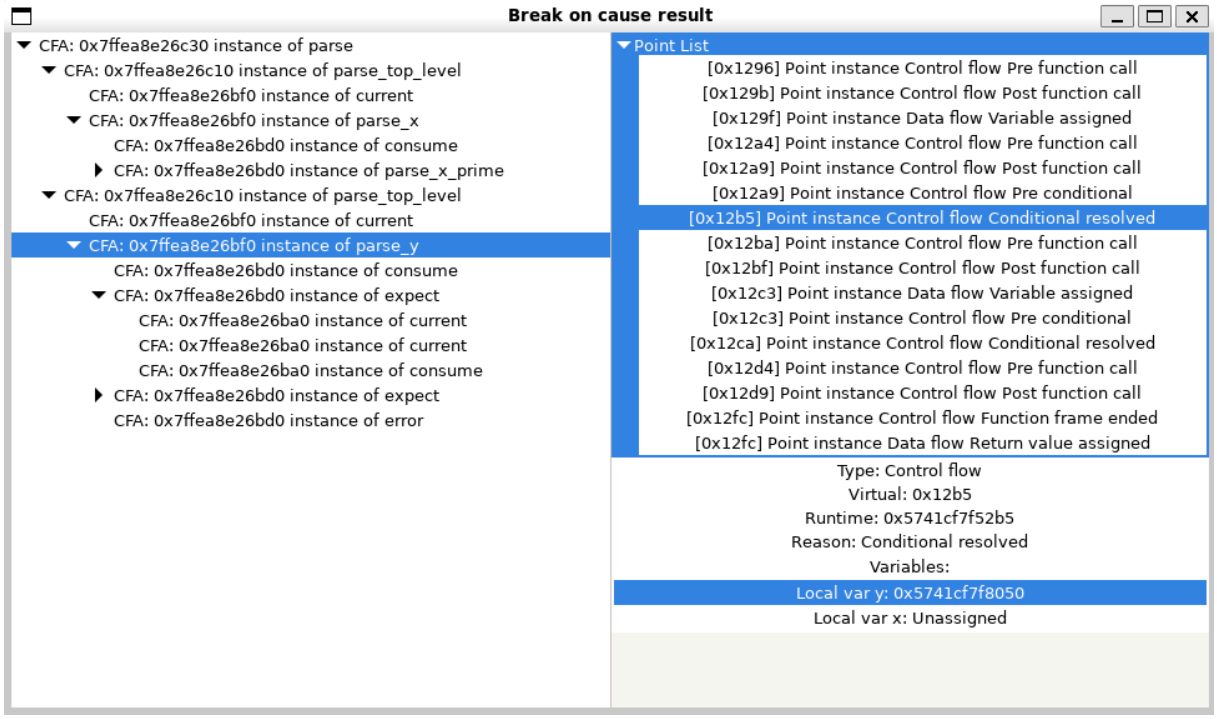

Figure 3: Example output of break-save

Code 20 shows an example output of the break-save feature, here on the left side is the stack frame tree. On the right is a list of the points of the selected frame, each displays it's virtual address, type, and reason. The selected point's information is visible at the bottom, along with the value of the variables at this part of execution.

9 Break on cause

Break-on-cause is designed to stop execution at the earliest point when it can guarantee that some variable will resolve to some value later on. Earliest means both being the deepest point in the call stack and the earliest point in the execution of that function.

The data flow in a program such Code 22 is fairly simple, FAIL and SUCCESS values are typically in the lowest level of calling and then just propagate back up. This requires some compiler generated

information that can describe the dependency of values, such as in this case res, allowing the debugger to place breakpoints in the required positions and then evaluate. If it can guarantee that something would occur later, which in this case is that res would equal FAIL, then it can break at some early point in execution.

```c
int parse() {
  ...
  while (stream_has_token()) {
    const int res= parse_top();
    if (res == FAIL) return res;
  }
  return SUCCESS;
}
```

```c
int parse_top() {
  Token* c= current();
  switch (c->type) {
    case TYPE_A: return parse_a();
    case TYPE_B: return parse_b();
    case TYPE_C: return parse_c();
  }
  return FAIL;
}
```

Code 22: Common parsing function overview

The initial design described this as a dependency, as in res depends on the expression parse_top_level() which depends on the function parse_top_level, which depends on four different return statements which depend on a conditional on c.type which depends on c which depends on consume() which depends on the function consume, etc.

There are however, two different types of dependency flow to consider with this; control flow and data requirement flow. The control flow is important because the debugger needs to put breakpoints in positions before they execute, but the data requirement can sometimes flow opposite to this.

```c
Token* c= current();
if (c == NULL) return FAIL;
```

Code 23:

The control flow of Code 23 goes from line 1 to 2, and the debugger will need to place a breakpoint on line 1 before it reaches line 2. The data requirement flow however flows from line 2 to 1, if we are wanting to break on the cause of the value FAIL then the conditional c == NULL places the data requirement of c being NULL which then means that the return value of current() must also be NULL.

9.1 Design

break-on-cause has three components, the first is the compiler generated information, this forms a graph of dependencies that describe how target values should spread through the program. The second is the debuggers instances of these entries, which I'll call markers markers use the compiler information to place breakpoints and callbacks and track their target value, these also form a graph through having a pointer to their previous marker. However, unlike the compiler information these represent the actual execution and so you can have markers with the same compiler entry at different points in the execution trace through recursion.

```c
int funcA() {
  return funcA() == 0;
}
```

Code 24: Example recursive function

DependencyTree

```
compiler entry tree:
DEP_HEADER id: 0
  DEP_EXPR id: 1 OP: OP_EQ TYPE: TYPE_LEFT
    DEP_LINEAR id: 2 LINK to id 0
    DEP_LINEAR id: 3 CONST 2
```

Dependency 1: Dependency tree for Code 24

DependencyTree

```
DEP_HEADER id: 0 target value: 1
  DEP_EXPR id: 1 target value: 1
    DEP_LINEAR id: 2 target value: 0
      DEP_HEADER id: 0 target value: 0
        DEP_EXPR id: 1 target value: 0
          DEP_LINEAR id: 2 target value: NOT 2
        DEP_LINEAR id: 3 DEAD
    DEP_LINEAR id: 3 DEAD
```

Dependency 2: Marker tree for instance of Code 24

The final component is the Stack tree, this is what records the stack frames when a frame ends before break-on-cause can halt. This also takes instances of the function arguments at the start of the frame, and the function variables at the end.

9.1.1 Compiler generated information

This section describes the format for the compiler generated information needed to support break-on-cause. At a high level there will be information for every function, and every local variable in the program, as well as the addition of a DWARF DIE data to link the local variables to their corresponding data. In addition the start address and all possible end addresses of each function are required as well, these however must be when the stack is still alive, and so the start address should be after all function arguments have been stored, and the end addresses should be before the stack pointer is overwritten.

Element	Description
unit length	Total size of the break-on-cause information
version	
offset_size	The size of offsets in the tables
sub_info_offset	Offset from the start of the information to the subroutine information table
dep_info_offset	Offset from the start of the information to the dependency information table

9.1.1.1 Subroutine information

As mentioned information on the start and end of subroutines is needed, these will be stored as a sequence of ULEB128s, with a terminating zero.

```
[SubStart], [SubEnd0], [SubEnd1], [0]
```

To index this information from a subroutine there can be an additional field added to the DIE information, DW_AT_sub_info which contains data of type DW_FORM_sec_offset, which can be used to offset into the debug_cause section's subroutine info table.

9.1.1.2 Dependency information

The first byte of the dependency table is reserved, allowing offsets of 0 to represent the end of a dependency chain.

There are six types of data entries

- DEPTYPE_HEADER
- DEPTYPE_LINEAR
- DEPTYPE_EXPR
- DEPTYPE_SPLIT
- DEPTYPE_COND
- DEPTYPE_COLLECT

All data entries contain three pieces of information todo: should this explain dead and pass?

```c
typedef struct DEP_BASE {
    DEPTYPE type;
    uint8_t dead: 1;
    uint8_t pass: 1;
} DEP_BASE;
```

9.1.1.3 Header dependencies

The header entry gives information to handle hitting a new function it contains two more pieces of information, the start address of the function and the offset to it's next entry.

```c
typedef struct DEP_HEADER {
    BASE
    ADDR addr;
    OFFSET link_offset;
} DEP_HEADER;
```

9.1.1.4 Linear dependencies

Linear dependencies have two forms, the first is a value entry which could be a constant value or a value at some fixed location e.g. a register, a register offset, a memory address, this is used when there is an operand e.g. a literal or variable. The second is a call entry which describes function calls.

Linear dependencies are one of the two 'hittable' dependencies, the other being the expression. These contain some shared information namely, an address, address offset, and a result location. This information allows the debugger to stop before some calculation pass on it's requirements to the link and then set a breakpoint at the offset which when hit can then evaluate the result at the location and see if it has reached it's required value. For call entries this means breaking before a function call, and creating the marker for the link, which would be a HEADER, and then setting a breakpoint on the offset which would be the instruction after the call and then evaluating the RAX register to check against the value needed.

```c
typedef struct DEP_HITTABLE {
    BASE;
    ADDR addr;
    OFFSET value_addr_offset;
    Location result_loc;
} DEP_HITTABLE;
```

```c
typedef struct DEP_LINEAR {
    HITTABLE_BASE
    OFFSET link;
} DEP_LINEAR;
```

9.1.1.5 Expression dependencies

Expression dependencies represent binary operators, they are also 'hittable' entries as they break before at the start and then propagate the value to their left and right entries. The address offset is to the assembly line where the result of the operation will exist.

```c
typedef struct DEP_EXPR {
    HITTABLE_BASE

    OPERATOR op;
    TYPE type;
    bool repeated;

    OFFSET left;
    OFFSET right;
} DEP_EXPR;
```

The type of the expression describes which of the operands can be traced. They are based on the following control flows.

left		right	TYPE
constant	OP	constant	TYPE_LEFT
x	OP	constant	TYPE_LEFT
constant	OP	x	TYPE_RIGHT
f()	OP	x	TYPE_LEFT
x	OP	f()	TYPE_RIGHT
x	OP	y	TYPE_BOTH

```c
typedef enum TYPE {
    TYPE_LEFT,
    TYPE_RIGHT,
    TYPE_BOTH,
    TYPE_NONE
} TYPE;
```

LEFT means that the value of the expression only depends on the left entry, and that the right is value available at the start of the expression. RIGHT is the same but only depending on the right and left being available. These types allow passing the value as the left/right's target value. BOTH means that both operands are calculated earlier in the control flow and so the target value is not available to propagate down. This can however be alleviated in some situations by moving the start of the expression to a point where one of the operands has been calculated but the other has not. Consider for example the following code

```c
int x,y;
x= a();
y= b();
return x == y;
```

```nasm
0x1e: call a
0x23: mov DWORD PTR [rbp-4], eax
0x26: call b
0x2b: mov DWORD PTR [rbp-8], eax
0x2e: mov eax, DWORD PTR [rbp-4]
0x31: cmp eax, DWORD PTR [rbp-8]
```

Here the expression x==y has two operands that are calculated earlier and so if the expression started at 0x2e it would not be able to give y a target value as y has already been calculated. Placing the start at 0x26 means that y has yet to be evaluated and the type becomes TYPE_RIGHT.

9.1.1.6 Split dependencies

Split dependencies are used for two parts, mainly in describing the different return points of a function, to describe that the value of a function call can depend on all exit points. Secondly, to describe the different points that a variable is assigned a value. For example:

```c
1 int res= SUCCESS;
2
3 if (t1() == FAIL) {
4     res= FAIL;
5 }
6
7 ...
8
9 return res;
```

This would be a split dependency to the conditional and to the res= SUCCESS. The reason for doing this is that we want to break as early as possible and so we can make the res= SUCCESS line, which will be the value of res if the conditional fails, into a LINEAR dependency with an address offset to point to the end of the conditional and a location of the local variable res (most likely some offset from the base pointer). This would mean that once the conditional ends the value of res is checked against the target value which was given by the split dependency and if it evaluates to the target we can break before all the code in '...'.

9.1.1.7 Conditional dependencies

Conditional dependencies describe the conditional control flow, they simply have a conditional link and a value link, as well as a boolean which is a simpler version of expression's type. In this case the conditional is the left and it is only dependant on it if the right value is a constant, otherwise the result of a conditional is only dependant on it's value. Similar to expressions we can solve control flow such as

```c
1 int x= f();
2 int res= parse();
3
4 if (x) return res;
```

By placing the start of the conditional after x has been evaluated on line 1 but before parse has been called, this allows the debugger to decide if res can halt if it reaches the target value, being if x resolves to truthy. If the conditional is on the actual conditional line then the value of res will have been evaluated already and so the debugger could only break on this line.

9.1.1.8 Collect dependencies

Consider the following program

```c
1 int res= parse();
2 if (res != SUCCESS) return res;
```

This creates a dependency on a conditional where res is used in the condition and the result. If the target value for this was any value not 1 then there are two requirements on res, that it is not 0

(SUCCESS) and that it is not 1. The Collect dependency combines target value requirements before passing it down to it's link.

```c
typedef struct DEP_COLLECT {
    BASE
    ULEB128 connections;
    OFFSET link;
} DEP_COLLECT;
```

9.1.2 Control flow

9.1.2.1 Loops

One previously unexplained variable in the DEP_EXPR entries is the repeatable boolean, this is used for the cases where expressions are within loops. This is because there are certain restrictions on when we can break when in a loop. The code in Code 25 shows a while loop that and's the result of parse with it's existing value. If the target value of this function is true then the dependency tree would propagate this to `res && parse()` being true, however parse should be dead in this case, because even if this instance is true, it does not mean that res will evaluate to true in the end. This would be true if the while loop was not there. The repeatable boolean is therefore there to differentiate between these two instances and allow the debugger to turn paths 'dead' when needed. This does mean that in this case the debugger cannot break in any of the parse paths, it can only save their stack frames and then break on `return res;` when it is sure of the value.

```c
int res= true;
  while (x) {
    res= res && parse();
  }
return res;
```

Code 25: Control flow: While loop example

9.2 Limitations

9.2.1 Pointers

Pointers introduce a great amount of indirection, which makes tracking data flow much more difficult. If the compiler can guarantee that a pointer is in fact a reference instead of a pointer then all instances of the pointer can be treated exactly the same as if the base value was used.

```c
int main() {
  int a= 0;
  int* b= &a;

  *b= funcA();

  return a;
}
```

Code 26: Pointer as reference example

```
DEP_HEADER main_header
  DEP_LINEAR return a;
    DEP_SPLIT
      DEP_LINEAR funcA();
      DEP_LINEAR int a=0;
```

Dependency 3: Dependency tree for Code 26

9.2.2 Function arguments

Currently target value propagation has to stop before a function call, there are a couple reasons for this. Firstly, an earlier call to the same function would start evaluating its result.

```c
int func() {
```

```
  for (int i= 0; i < 10; i++) funcA();
  return funcA();
}

int funcA() {
  return funcB();
}
```

The first layer of this could be solved by adding a return address value to the dependency entry, this would differentiate between the two calls. However this rule breaks at the next level, because if the target value for return funcA has been propagated then the target value for return funcB has been as well and the return address for all instances of funcB is the same, even if funcA was called at different points.

Secondly, exploring every possible function call down a possible call chain to propagate a target value would be extremely expensive, it would have to explore every possible execution path, currently function calls can prune this tree, as they may never be hit.

This restriction therefore makes it difficult to include function arguments in the break-on-cause

c

```
int main() {
  TYPE type= TYPE_NONE;
  int res= parse(type);
}
int parse(TYPE type) {
  if (type == TYPE_NONE) return FAIL;
  ...
}
```

Code 27: Function argument example

Code 27 shows an example program where the earliest point that we know res will result in FAIL is

9.3 Further development

9.3.1 Variable timeline

Currently there is a stack frame tree which shows the timeline of which were called first, and information on their variables and args. One way to further develop break-on-cause given its current functionality would be to add onto the dependency entries a list of the variables that are about to be altered. The reasoning for this is that when the entry is hit an instance of the variable can be taken and stored in the frame, and at the end a timeline of the values of the variable throughout the function can be shown, as well as a timeline of the markers and these variable values to see the control flow and data changes. This would bring break-on-cause closer to time-travel debugging while not having to provide all the features of full memory saving and restoring frames.

10 Conclusion and reflections

10.1 Contributions

This project has developed the core parts of an extensible debugger, including key features such as source level stepping, breakpoints, a stack trace, and register view. As well as developing an instruction set description language that allows for easy development of new disassemblers. The project explores two types of data flow analysis in the form of break-save and break-on-cause, with the latter bringing a feature uncommon in any debugger.

10.2 Project management

This project followed a gantt chart that was designed at the beginning and refined during the interim report. This was in combination with a series of work packages which aimed to group some of the tasks into more cohesive packages. The first half of the project during the Autumn term experienced a slower pace than I had anticipated or planned for, this was mainly due to my 70/50 split across semesters. This is what lead to the updated gantt chart and work plans where tasks such as the DIE parsing, ui views, and source level stepping were moved into the spring term. Longer sub-tasks like ISDL were also extended further into spring. Overall this worked well for the project, I had much more time in the spring term to focus on Anura, and it allowed for previously unplanned aspects of the project to be explored and implemented.

10.3 Reflections

Throughout development of this project it has evolved significantly, the original plan did not include any data flow analysis, or pseudo-compiler-generated information, being break-save and break-on-cause. These were developed in the later third after an insightful meeting with my supervisor. I do believe that these are some of the strongest aspects of the project, especially break-on-cause which is not a feature found in any typical debugger. This did however lead to some other areas of the project that were planned having to be de-prioritised and eventually incomplete, these are all UI tasks, specifically a register view, a stack frame view, and a symbol view.

Another part of the project that fell short is rigid support for multi-compilation unit programs, i.e. programs that contain more than one source file. Many components of DWARF split up CU information, for example the line number program, the Debug Information Entries, and the frame information program. The main issue Anura currently has is that it does not clear and repopulate this information based on a change in CU, in fact it does not currently ask for a distinguishing reference to the correct CU. Most areas use the main CU reference which points to the CU containing the main function. This was not a consideration in my original plan, and during development most testing programs were single file programs. Although this does limit Anura's current usefulness it does provide a base for each of these parts which should be relatively easy to include CU information and have each clear and recalculate it's information (or even cache) for new CUs. This is one of the reasons that Anura cannot currently debug itself, the other being that multi-threading is not currently supported.

The graphical version of Anura is currently restricted to opening a new instance of a target program, it cannot attach to one. This would require a view into the current running processes and some UI view, this was not implemented in the end mainly as I did not believe it added much value to the project.

10.4 Future work

Anura is not several things, but it is mostly most importantly not a finished product, while many of the features of a debugger have been implemented the overall cohesion of them needs to be developed in the future.

There are a few more features that appear regularly in debuggers that Anura does not yet support. One of these is conditional breakpoints, which allow the user to enter some condition that when the breakpoint is hit, it will only break the program for the user if the condition evaluates to true. This would require creating an expression parser and evaluator for Anura, and is a feasible extension for the project, it was omitted from the plan for time reasons, but with information such as the virtual variables now available there is a base on which to build it.

Another missing feature is watchpoints, these are breakpoints that break when data is read/written to. Hardware watchpoints require little implementation as they just set different values in the debug control register and the CPU will break on read/write. Software watchpoints however require the program to essentially single-stepping the entire program's execution and testing if the value has changed at each point. This was not a planned feature for Anura, however there should be a large base implementation for this to be implemented.

Furthermore there is lots of development to support multithreaded applications, this is the part where Anura provides the least amount of base support - being none. This would require some thread context for breakpoint information, and state. Anura uses state when doing multi-hit effects like step-over, step-into, break-on-cause, and break-save, where they may hit breakpoints and halt the program multiple times but still be doing the same action.

Overall Anura has a lot of room to grow into, but has provided a base that should make future features much easier to implement.

10.4.1 LSEPI

This project created a new application; a debugger, because of this I have considered if this project should be commercialised or free, closed source or open source. There are two main reasons I have chosen free and open source, firstly I believe that because this project is built upon so many other FOSS projects; Linux, GTK4, ELF, and DWARF, that it should be placed in the same position to benefit others. Secondly the alternatives to Anura are mostly FOSS; mainly GDB and LLDB, and positioning as an alternative to these is not feasible if it is more restrictive.

As part of making the project open source I will need to choose a license, the three I considered are the MIT License, GNU GPLv3, and the Unlicense. The GNU GPLv3 license would require derivative works to be under the same license, state the changes, and to disclose the source [32]. The MIT license requires only that the copyright notice be included with the material [33]. Finally the Unlicense has no restrictions on usage [34]. All of these licenses allow for commercial and private use, along with distribution and modification of the material. I have selected the GPLv3 license as it has broad allowance for people to modify the work while making sure that any derivatives are able to stay free and open source.

Debuggers mainly serve developers as their usefulness increases exponentially with debug information; however they can still be used on stripped binaries and there exists the possibility that Anura could be used in, for example, reverse engineering with the intent to infringe on IP/copyright laws or breach software licenses. These are the main legal concerns for the project, however these are limited in scope and unlikely to be used as there are better tools for this such as Ghidra or IDA.

11 Declaration of AI usage

I confirm that this dissertation is my own original work. I have not used Artificial Intelligence (AI) tools to generate academic content or ideas.

12 Bibliography

Bibliography

[1] "Info World: News for microcomputer users," 1981, [Online]. Available: https://books.google.co.uk/books?id=JT0EAAAAMBAJ&pg=RA1-PA33&redir_esc=y#v=onepage&q&f=false

[2] "Letter from J. Robert Oppenheimer to Dr. Ernest Lawrence." [Online]. Available: https://web.archive.org/web/20191121001830/https://bancroft.berkeley.edu/Exhibits/physics/images/bigscience25.jpg

[3] “PDP-1 program library.” 1964. [Online]. Available: https://www.bitsavers.org/pdf/dec/pdp1/DIGITAL-1-3-S_ddtDesc_Aug64.pdf

[4] D. E. Corporation, “PDP 11 handbook.” 1969. [Online]. Available: https://gordonbell.azurewebsites.net/digital/pdp%2011%20handbook%201969.pdf

[5] “dbx User Guide.” 2003. [Online]. Available: https://www.infania.net/misc1/sgi_techpubs/techpubs/007-0906-140.pdf

[6] “GDB.” [Online]. Available: https://sourceware.org/gdb/

[7] “GDB remote debugging.” [Online]. Available: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Remote-Debugging.html#Remote-Debugging

[8] Microsoft, “Debug Adapter Protocol.” [Online]. Available: https://microsoft.github.io/debug-adapter-protocol//

[9] “LLDB.” [Online]. Available: https://lldb.llvm.org/

[10] “GDB supported targets.” [Online]. Available: https://sourceware.org/gdb/wiki/Systems

[11] “GDB target description files.” [Online]. Available: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Target-Descriptions.html#index-target-descriptions

[12] “LLDB supported targets.” [Online]. Available: https://lldb.llvm.org/#platform-support

[13] “LLDB Command Structure.” [Online]. Available: https://lldb.llvm.org/use/tutorial.html

[14] “WinDbg engine.” [Online]. Available: https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/debugger-engine-overview

[15] “Komodo's Assembler-Disassembler description language CHUMP.” [Online]. Available: https://brej.org/chump/

[16] “GHIDRA's target description language SLEIGH.” [Online]. Available: https://ghidra.re/ghidra_docs/languages/html/sleigh.html

[17] “Example CHUMP programs.” [Online]. Available: https://brej.org/chump/sample.chump

[18] “SLEIGH's processor definitions.” [Online]. Available: https://ghidra.re/ghidra_docs/languages/html/sleigh_definitions.html

[19] “P-Code Reference Manual.” [Online]. Available: https://ghidra.re/ghidra_docs/languages/html/pcoderef.html

[20] J. Bailey and C. Nicholas, “Symbolic Execution in Practice: A Survey of Applications in Vulnerability, Malware, Firmware, and Protocol Analysis.” [Online]. Available: https://arxiv.org/abs/2508.06643

[21] T. Committee, “TIS ELF Specification.” 1995. [Online]. Available: https://refspecs.linuxfoundation.org/elf/elf.pdf

[22] D. D. I. F. Committee, “DWARF Debugging Information Format Version 5.” 2017. [Online]. Available: https://dwarfstd.org/doc/DWARF5.pdf

[23] L. foundation, “EH Frame information.” [Online]. Available: https://refspecs.linuxfoundation.org/LSB_1.3.0/gLSB/gLSB/ehframehdr.html

[24] “GTK4 Platforms.” [Online]. Available: https://www.gtk.org/docs/installations/

[25] “Linux waitpid function.” [Online]. Available: https://linux.die.net/man/2/waitpid

[26] “Windows debug event function.” [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/debugapi/nf-debugapi-waitfordebugevent

[27] “YAMA security module.” [Online]. Available: https://docs.kernel.org/admin-guide/LSM/Yama.html

[28] G. D. Team, “GTK4 thread requirements.” [Online]. Available: https://docs.gtk.org/gtk4/section-threading.html

[29] I. Corporation, “Intel® 64 and IA-32 Architectures Software Developer's Manual: Instruction Set Reference, A- Z.” 2026. [Online]. Available: https://cdrdv2.intel.com/v1/dl/getContent/671110

[30] I. Corporation, “Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4.” 2025. [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

[31] “Ptrace man page.” [Online]. Available: https://man7.org/linux/man-pages/man2/ptrace.2.html

[32] “GNU GPLv3 License.” [Online]. Available: https://www.gnu.org/licenses/quick-guide-gplv3.html

[33] “MIT License.” [Online]. Available: https://opensource.org/license/mit

[34] “Unlicense License.” [Online]. Available: https://unlicense.org/